pdfulator, a Markdown to PDF converter

Tom Gidden <tom@gidden.net>

August 2026

v.3

https://github.com/tomgidden/pdfulator

Introduction

`pdfulator` converts Markdown to PDF, and tries to be the one command you need rather than a pipeline you have to assemble yourself.

There is more than one good way to get from Markdown to a paginated PDF, and they disagree about interesting things — how much Markdown they understand, how faithfully they paginate, whether they need a browser, whether they render the same on your laptop as on a build server. So `pdfulator` does not pick one. It provides several **engines**, and one interface over all of them:

```
pdfulator report.md                      # the default engine
pdfulator --engine pandoc-pagedjs report.md
pdfulator --list-engines                 # what's installed
```

Whichever you choose, the command's own behaviour does not change. It plans the job — which files, where the output goes, whether it needs rebuilding, whether to watch for changes — and resolves the theme. The engine only ever converts one document.

Engines

--engine=	Docker	Description
`vivlio`	no	Default: *markdown-it* + *vivliostyle*, with local JS and Chrome-based browser for rendering
`vivlio-docker`	yes	As `vivlio`, but contained in *Docker* for isolation, and *Puppeteer* for rendering
`pandoc-pagedjs`	yes	*Pandoc* + *PagedJS*, and rendered with *Puppeteer*
`pandoc-xslt`	yes	*Pandoc* to *DocBook 5*, then *XSLT* and *Apache Fop* (*XSL-FO*) (deprecated)

`vivlio` is the default and runs on your own machine, so it renders with your own fonts — which is why it is the default rather than the containerised one. It uses *markdown-it*, *Vivliostyle*, *Mustache* and *Puppeteer*, on *bun* or any *node* or *deno* you already have, driving a *Chromium*-based browser.

`vivlio-docker` is the same pipeline in a container: nothing installed on the host, no runtime, no browser, no dependencies. The trade is fonts — a container has only the fonts baked into it, so a document leaning on something installed locally renders differently. Reproducible-everywhere and looks-like-my-laptop are different wants, and only you know which you have.

`pandoc-pagedjs` is v1's pipeline, rebuilt: *Pandoc* parses, *Paged.js* paginates, Chromium prints. Kept because it is genuinely different rather than merely older — Pandoc reads far more Markdown than markdown-it does, bringing definition lists, footnotes and tables that `vivlio` has not got.

`pandoc-xslt` is the oldest pipeline still standing, from the DocBook and XSL-FO days, and is deprecated from the day it ships. XSL-FO is a dead standard and FOP is its last maintained implementation — but the typesetting is genuinely better than the browser engines', and fifteen years of accumulated layout work is not something to throw away. It is here because that output is worth keeping, not because anyone should start a new document with it.

The container engines need only Docker. `pdfulator` fetches the image the first time you use one, and says so:

```
pdfulator --engine vivlio-docker report.md
pdfulator --prepare              # or fetch it now, before you need it
```

Your choice is remembered, so `--engine` is something you set once rather than type every time.

Installing

Install with:

```
curl -fsSL https://pdfulator.app/get | sh
```

That downloads the current release into `~/.local/share/pdfulator` (override with `PDFULATOR_HOME`) and puts a `pdfulator` command in `~/.local/bin` (`PDFULATOR_BIN`). The download is checksum-verified, and reinstalling over an existing copy keeps your themes.

If it has a terminal to ask on, it then walks you through the two remaining choices — which JavaScript runtime to use, and which browser to render with — listing what it found on your machine and defaulting to the sensible answer. Run `pdfulator --install` any time to change your mind.

Piped somewhere without a terminal (a Dockerfile, a provisioning script), it installs and tells you to run `pdfulator --install` when convenient, rather than guessing. Or answer up front:

```
curl -fsSL https://pdfulator.app/get | sh -s -- --install-runtime --
install-browser
```

Nothing is fetched without you asking. pdfulator runs on bun — it will use an existing node or deno if you have one, but bun is the only runtime it will download for you — and renders with a Chromium-based browser.

It can use one of your existing Chromium-based browsers, and even try to locate them; or you can tell it to install a minimal browser (`chrome-headless-shell`) for its own use. `pdfulator --install` asks, but you can also set it directly:

```
pdfulator --browser XXX
```

where `XXX` is one of:

- `auto`: locate an installed Chromium browser and remember it
- `find`: list located browsers for you to choose
- `install`: install a minimal browser inside the pdfulator install
- a path to a browser executable.

Either way, it remembers your choice.

Then:

- `pdfulator README.md` (generates `README.pdf`)
- `pdfulator README.md foo.pdf` (generates `foo.pdf`)
- `pdfulator .` (converts every `*.md` in the current folder)
- `pdfulator docs/ pdfs/` (converts `docs/*.md` into `pdfs/`, creating it if needed)
- `pdfulator - < foo.md > foo.pdf` or `cat foo.md | pdfulator - > foo.pdf`

and you can uninstall with `pdfulator --uninstall`

Options

```
pdfulator [options] input.md [output.pdf]   convert one file
pdfulator [options] dir/ [outdir/]          convert every *.md in
a directory
pdfulator [options] -                       stdin → stdout
pdfulator --watch [options] dir/ [outdir/]  rebuild on change
```

There are two shapes, each taking an optional destination: a file becomes a file, and a directory becomes a directory. Whether the destination is a file or a directory follows from the source, so nothing changes meaning depending on

what happens to be on disk — `pdfulator *.md` is not a supported form precisely because its meaning would depend on how many files the glob matched. Use a directory to convert many files at once.

A destination is only overwritten if it is genuinely a PDF (checked by content, not by name) or doesn't exist yet. An output that is already newer than its source is left alone, and says so.

Option	Meaning
`-t`, `--theme <name|path>`	Theme to use
`--css <file>`	Extra stylesheet, applied after the theme
`--font-fallback`	Use standard PDF fonts when one can't be had
`-e`, `--engine <id>`	Engine to use (remembered)
`--list-engines`	Show the installed engines
`-d`, `--debug`	Keep the intermediate files
`-v`, `--verbose`	Verbose output
`-w`, `--watch`	Watch a directory for changes
`-h`, `--help`	Show help

And for setting things up:

Option	Meaning
`--install`	Choose a runtime and browser, asking if there's a terminal
`-b`, `--browser auto|find|install|<path>`	Choose the rendering browser (remembered)
`--install-runtime`	Download a private copy of *bun* if none is installed
`--prepare`	Fetch what the chosen engine needs, now rather than on first use
`--setup-status`	Report what is still needed
`--update`	Install a newer release, if there is one
`--version`	Report the installed version
`--uninstall`	Remove pdfulator, keeping anything you added or edited

Nothing is downloaded or launched without you asking: pdfulator will explain what it needs and wait rather than picking a browser or fetching a runtime on your behalf.

Updating

```
pdfulator --update          # asks first
pdfulator --update --yes    # doesn't
pdfulator --update --check  # exits 0 if an update is available, 1 if not
```

An update keeps your themes, your chosen browser and runtime, and the private *bun* or Chromium if you have one — only the application itself is replaced. Dependencies are reinstalled on next use if the release changed them.

`PDFULATOR_VERSION` pins a particular release, which is also how to go back:

```
PDFULATOR_VERSION=v2.0.0 pdfulator --update --yes
```

A build made from a checkout (`make install-local`) is left alone rather than being replaced by a release, since that would discard whatever you were working on. `--force` overrides that.

What it installs, and where

Everything lives under `$PDFULATOR_HOME` (`~/.local/share/pdfulator` by default), plus the wrapper itself in `$PDFULATOR_BIN` (`~/.local/bin`):

Path	Contents	Size
`lib/`, `theme/`	The command's shared layer	small
`themes/<name>/`	The bundled themes, and any you add	~3MB
`engines/<id>/`	One converter each — see `--list-engines`	small
`engines/<id>/node_modules/`	An engine's dependencies, if it has any	~30MB
`fonts/`	Fonts a theme downloaded, shared between themes	varies
`cache/`	Themes prepared for an engine; rebuilt when needed	small
`bun/`	Private *bun*, only if you asked for one	~60MB
`chromium/`	`chrome-headless-shell`, only if you asked for one	~193MB

Dependencies are per-engine and installed on first use, so you only pay for the engines you actually run. The same is true of container images and of any fonts a theme fetches: nothing arrives until something needs it.

`pdfulator --uninstall` removes all of it — including the `pdfulator` command itself — except files you have added or modified. Your themes and any edited stylesheets are kept, and it tells you what it left behind; if there's nothing to keep, the directory goes too.

Customisation and development

The wrapper runs from a checkout exactly as it does when installed, finding `lib/`, `engines/` and `themes/` beside itself:

```
./pdfulator.sh README.md
```

What you need depends on which engine you are working on, and nothing more: the container engines need only Docker, and `vivlio` needs bun (or node, or deno) and a Chromium-based browser. An engine's dependencies install on first use.

`--debug` keeps the intermediate files, which is usually what you want when adjusting a theme; `--watch` re-renders on every save:

```
./pdfulator.sh --debug --theme ./my_theme README.md
./pdfulator.sh --watch .
```

Everything is POSIX `sh` — no bashisms — because it has to run under `dash` on Debian as happily as under `zsh` on macOS.

```
make test-lib         # the shell suites: no browser, no Docker, ~90s
make test             # ...and the ones needing a browser or a tarball
make dist             # produces ./pdfulator.tar.gz, what CI publishes
make install-local    # ...and installs it, exactly as install.sh would
make install-source   # installs the working tree directly, skipping
the tarball
```

`install-local` is the faithful one: it installs the exact bytes a release would ship, which is what you want before cutting one. `install-source` skips the packing step for when that is what's in the way — editing a file in `lib/` and wanting the installed command to have it, or bisecting. It goes through `install.sh` either way, so both produce the same tree, manifest and all, and `--uninstall` works afterwards regardless of which you used. Directly:

```
PDFULATOR_SOURCE=/path/to/checkout sh install.sh
```

A source install takes its version from `git describe`, `-dirty` suffix included — which is what stops `--update` from offering to replace a work in progress with a release.

`make test-lib` is the one to run while working. It is a few hundred assertions over the job planner, theme resolution, the font cascade, engine dispatch and watch mode, and it needs nothing installed.

Themes

A theme is a directory. `pdfulator` ships two:

- **`default`** — standard PDF fonts (Times, Helvetica, Courier), sane resets, no layout. It needs no network and no downloads, which is what makes it the floor everything else stands on.
- **`classic`** — the "white-paper" look: TeX Gyre Pagella, Open Sans and Noto Sans Mono, with running headers and footers. Influenced in my youth by the original 1995 Java™ white paper and other documentation from Sun.

```
pdfulator --theme classic report.md
pdfulator --theme ./my_theme report.md
pdfulator --css tweaks.css report.md      # one-off, on top of the theme
```

A named theme is looked for in `./themes/<name>/`, `$PDFULATOR_HOME/themes/<name>/`, then `themes/<name>/` alongside the command, so a theme installed in the second is available anywhere. If it isn't found, `pdfulator` stops and says where it looked rather than quietly using the default — a typo would otherwise produce a perfectly plausible PDF in the wrong style, which you'd only catch by eye.

Writing one

```
my_theme/
  theme.conf                  name, description, and what it extends
  fonts.conf                  fonts, by role
  print.css                   styling for any engine
  fonts/                      font files, if it ships any
  stylers/vivliostyle/        styling for a particular renderer
  stylers/pagedjs/
  engines/pandoc-xslt/        ...or for one specific engine
```

Everything is optional. A theme is data throughout — nothing in it is ever executed, which matters because themes are meant to be shared.

Themes extend other themes. `theme.conf` names a parent:

```
name = palatino
description = Classic, but with a different body font
extends = classic
```

and supplies only its differences. CSS is concatenated parent-first, so a child's rules win by ordinary precedence. Fonts are inherited **per role**, so a theme changing only its body font keeps its parent's headings and monospace untouched — which means a theme can be one short file.

Fonts are declared, not linked. `fonts.conf` names them by role:

```
body.family = EB Garamond
body.source = local
body.face.400.normal.file = fonts/EBGaramond-Regular.ttf
body.face.700.normal.file = fonts/EBGaramond-Bold.ttf

heading.family = Figtree
heading.source = url
heading.face.400.normal.url    = https://example.org/Figtree-Regular.ttf
heading.face.400.normal.sha256 = a1b2c3...
```

`source` is `local` (a file in the theme), `url` (fetched once, cached under `$PDFULATOR_HOME/fonts/`, verified if you give a checksum) or `none` (the standard PDF fonts, which need no file at all).

The roles `body`, `heading` and `mono` are understood by every engine. That indirection is the point: your stylesheets refer to `var(--pdfulator-body)` rather than to a font by name, so changing the font is a `fonts.conf` edit and no CSS changes at all. `pdfulator` generates whatever the chosen engine actually needs from that one declaration — `@font-face` rules for the browser engines, an explicit font configuration for FOP.

A font that can't be had is an error, naming the role, the font and why:

```
Error: this theme needs a font it cannot get.
  role:   body
  font:   EB Garamond (local)
  reason: no such file: /path/to/my_theme/fonts/EBGaramond-Regular.ttf

Render anyway with standard PDF fonts:
  pdfulator --font-fallback ...
```

That is the whole reason for declaring fonts rather than just linking them. A font named in one place and missing from another is how a document renders in the wrong typeface for years without anyone noticing.

Per-engine styling. Different renderers present different DOMs, so a theme can hold CSS for a particular one under `stylers/<styler>/` — `vivliostyle`, `pagedjs` or `xsl-fo`. `stylers/` rather than per-engine because `vivlio` and `vivlio-docker` are the same renderer and would otherwise need two identical copies. `engines/<id>/` is there for the rarer case of a genuine engine-specific difference, and wins over both.

Logo

If the theme contains a `logo.svg`, the stylesheet can place it in the page margin. To position or size it, use a little custom CSS — and note the quirk that you have to set `content`, even though the main stylesheet already does:

```
@page {
  @top-right {
    content: string("");                    /* seemingly important */
    background-image: url(/theme/logo.svg);
    background-size: 108pt auto;            /* here's how to size the
image */
    /* control `height`, `margin-top` and `margin-right` appropriately,
but check
     * it doesn't crash into content on page 2 onwards.
     */
  }
}
```

The margins around it are not great yet — see the TODO.

Document metadata

To support the top front-matter in a Markdown file, you can include a YAML block at the top of the file delineated by `---` and `...` — this README begins with one. Alternatively, put it in a `.yaml` file beside the document (`report.md` and `report.yaml`), which keeps the Markdown clean and is picked up automatically.

Unfortunately, other Markdown renderers (notably *GitHub*) may include this as garbled nonsense in their output.

If this bothers you, you can include the YAML as a separate file next to the Markdown instead, eg. `foo.md` and `foo.yaml`.

Metadata entries

- `title` - The title of the document. If not included, the first top-level heading (`#`) in the document will be hoisted to the title. It seems weird to leave the title out of the bare Markdown, but this hoisting behaviour is a little unusual.

- `date` - A block containing `month` and `year` to be displayed in the front-matter.

- `revision` - A revision number, to be displayed in the front-matter and the footer.

- `copyright` - An optional copyright attribution to be included in the page footer. If set, it will be preceded by '©', the year (determined either from `date.year` or `year` in the metadata, or the current year), and then the attribution.

- `footer` - An optional text to be included in the page footer, added to the copyright if there is one.

- `pdfulator_features` - See below

Example metadata

```
title: Set this or just let it use the first `#`

date:
  month: September
  year: 2024
revision: First Draft
authors:
- firstname: Tom
  surname: Gidden
  email: tom@gidden.net
- name: D. C. O'Author
  affiliation: Institute of Documentarian Affairs
- firstname: Harold
  surname: L'astname
  affiliation: Institute of Documentarian Affairs

copyright: Tom Gidden & Institute of Documentarian Affairs
footer: Confidential

pdfulator_features:
- no_wide
- no_wide_pre
- shade_monospace
- strong_monospace
- narrow_monospace
```

`pdfulator_features`

The document metadata see above can include a `pdfulator_features` line or list that contains a few optional choices controlling formatting. These can be left in but ignored (ie. disabled) by prefixing them with `no_`, or just removing them.

These include:

- `wide` - Don't indent the main body text. This gives extra space, useful especially for pre- and code-blocks, but at the expense of the left margin.

- `wide_pre` - Don't *further* indent pre-formatted text blocks. Again, extra space, but less easily read.

- `shade_monospace`, `shade_pre`, `shade_code` - Use a light grey background for monospaced font material, or just block (`_pre`) or inline (`_code`) sections. This is to further distinguish from the main text.

- `strong_monospace`, `strong_pre`, `strong_code` - Use a bolder font for monospaced. By default it uses a lighter font to try to distinguish from body text, but this goes the other way.

- `narrow_monospace` - Use a narrower font for monospaced content, to try to get it to fit nicely on a page.

- `justify` - Use full justification for body text. I like this, but some of my designer friends say it's bad and ragged edge makes for better typography.

- `strong_href` -- embolden hyperlinks to make them stand out.

TODO

[X] *Themes*.

[X] *Built-in themes*. `default` and `classic` ship; `--theme` picks one.

[X] *Multiple files*. Handled by the directory form (`pdfulator src/ out/`) rather than a list of filenames, so an argument's meaning never depends on how many files a glob matched.

[X] *One-line installer*. `curl -fsSL https://pdfulator.app/get | sh`, with the wrapper distributed from CI rather than built from a checkout.

[X] *Testing of* `--watch`. Poll, `fswatch` and `inotifywait` branches all covered, including the self-triggering case where a rendered PDF looks like a change.

[ ] *More themes*, and a way to install one you didn't write.

[ ] *Tidy the margin-box logo*. It renders, but the spacing around it wants work, and it should be verified across all four engines rather than just the default one.

[ ] *TOCs*

[ ] *Better images*. Assets in the theme folder can be referenced, and remote URLs presumably work. Under Docker they still have to be mounted in, which is awkward. More thought needed.

[ ] *Improved layout*. `classic` renders differently under each engine, since its two stylesheets are two generations of the same file. Making them agree is its own job.

[ ] *Comprehensive support for the format*

[ ] *HTML*, *EPUB*, etc. The pipeline already produces HTML on the way to PDF, so exposing it should be straightforward. I'm just an old fart that likes neat A4 documents even if I never actually print them out.

[ ] *A* `remote` *engine*, so the work can happen on a server rather than on your laptop.

Any feedback, assistance or code contributions welcome.

History

I've had various DocBook or Markdown to PDF toolchains using XSL-FO, Apache FOP and other tech since the late nineties, usually named "docbot" as I used them in web and email services, Slack bots, etc.

There are a lot of projects called `docbot` and a lot called `md2pdf`. None of them do exactly what I want, though. Decent pagination was served by the DocBook XSL sheets, but HTML-based ones have been lacking. Most still do.

And using DocBook as an intermediary is a bad idea; while DocBook is richer than Markdown (and arguably a far better choice for software documentation) the element semantics aren't suitable for generic documents.

PagedJS now seems to make the HTML route a good option, being a capable polyfill for the print features of CSS3, allowing for running headers and footers and so on.

- v1.0: Released for a short time as "docbot"
- v1.1: Renamed to "pdfulator" and refactored to give a basic "--watch" mode. This is still a work in progress.
- v1.1.1: Themes
- v1.1.2: Preservation of work folder (now in `/work`), and minor styling for logos
- v2: Rewritten around markdown-it and Vivliostyle, with a one-line installer, a `--watch` that works, and themes.
- v3: Several engines rather than one pipeline, each behind the same interface; themes that inherit from each other and declare their fonts rather than linking them.

Licence

I've bundled those fonts purely for performance and simplicity: otherwise they either need downloading on each invocation, or caching somehow between Docker runs, leaving junk on the host machine. I hope that's okay within the terms of those licences. A theme you write yourself can equally well fetch its fonts rather than ship them — see `fonts.conf` above — in which case they are cached once under `$PDFULATOR_HOME/fonts/` and shared between themes.